Oracle PL/SQL Complete Mastery Tutorial

From Absolute Beginner to Advanced Database Programming

Placeholders • Variables • Constants • Literals • Scope • Control Flow • Records • Cursors • Exceptions • Collections • Procedures • Packages • Triggers • Dynamic SQL • Bulk Binding • Performance Tuning

Two tutorials merged into one structured course: the beginner-friendly chapter-by-chapter tutorial and the full A-to-Z mastery guide. Nothing was removed; overlapping explanations were combined and corrections were applied where the two versions differed.

Table of Contents

PART I - Foundations (Beginner) 4

Chapter 1 - Understanding SQL and PL/SQL from Scratch . . . 6
Chapter 2 - The Structure of a PL/SQL Block . . . 10
Chapter 3 - Placeholders . . . 13
Chapter 4 - Variables . . . 14
Chapter 5 - The %TYPE Attribute . . . 21
Chapter 6 - PL/SQL Constants . . . 23
Chapter 7 - PL/SQL Literals . . . 26
Chapter 8 - Scope of PL/SQL Variables . . . 29
Chapter 9 - SQL and Mathematical Operations in PL/SQL . . . 32
Chapter 10 - Running SQL Inside PL/SQL (DML) . . . 34

PART II - Intermediate 37

Chapter 11 - Conditional Logic: IF and CASE . . . 39
Chapter 12 - Loops and Iteration . . . 41
Chapter 13 - Records: Managing Entire Rows . . . 43
Chapter 14 - Cursors: Handling Multi-Row Queries . . . 44
Chapter 15 - Exception Handling: Catching Errors . . . 46
Chapter 16 - Collections: Arrays in PL/SQL . . . 48

PART III - Advanced 50

Chapter 17 - Procedures and Functions: Reusable Code . . . 52
Chapter 18 - Packages: Organizing Code . . . 54
Chapter 19 - Database Triggers: Event-Driven Automation . . . 56
Chapter 20 - Transactions: COMMIT, ROLLBACK and SAVEPOINT . . . 58
Chapter 21 - Dynamic SQL: Queries Built at Runtime . . . 59

PART IV - Professional 60

Chapter 22 - Bulk Binding: Minimizing Context Switches . . . 62
Chapter 23 - Advanced Cursors: Dynamic Results and Locking . . . 63
Chapter 24 - Performance Tuning . . . 64
Chapter 25 - Advanced Capabilities and Oracle Utilities . . . 65
Chapter 26 - Security and Professional Coding Practices . . . 67
Chapter A - Final Cheat Sheet . . . 68

How to Use This Tutorial

This document merges two PL/SQL tutorials into a single learning path. Start at Chapter 1 and read in order: every chapter only relies on ideas introduced in earlier chapters. Run every example yourself in **Oracle SQL Developer** or **SQL*Plus** - typing the code is the fastest way to learn.

Colour-coded boxes

Box	Meaning
NOTE	Extra information worth remembering.
KEY IDEA	The core concept of the section - make sure you understand this one.
WARNING / COMMON ERROR	Mistakes that beginners make and the errors Oracle will show you.
TIP	Good practice used by professional developers.
PRO ADDITION	Material added beyond the original lecture slides.
CORRECTION	A place where the two source tutorials differed or a statement was made more precise.

Roadmap

LEARNING PATH

```
BASIC                 INTERMEDIATE              ADVANCED
-----                 ------------              --------
Block Structure  -->  Control Flow         -->  Procedures / Functions
Variables        -->  Records              -->  Packages
Constants        -->  Cursors              -->  Triggers
Literals         -->  Exception Handling   -->  Dynamic SQL
                 -->  Collections          -->  Bulk Binding / Performance Tuning
```

Level	Chapters	What you will learn
Beginner	1 - 10	SQL vs PL/SQL, blocks, placeholders, variables, data types, %TYPE, constants, literals, scope, operators, SQL inside PL/SQL
Intermediate	11 - 16	IF / CASE, loops, records, cursors, exception handling, collections
Advanced	17 - 21	Procedures and functions, packages, triggers, transactions, dynamic SQL
Professional	22 - 26	Bulk processing, advanced cursors, performance tuning, autonomous transactions and Oracle utilities, security and professional coding practices

> **NOTE** This tutorial uses Oracle Database syntax. Examples that use tables such as `EMP` and `DEPT` assume the classic Oracle sample (SCOTT) schema is installed. Other examples use an illustrative `employees` table that you can create yourself (see the sample-data box in Chapter 1).

PART I

Foundations (Beginner)

Understand what PL/SQL is, how a block is built, and how to store, name and protect your data: placeholders, variables, constants, literals and scope.

CHAPTER 1

Understanding SQL and PL/SQL from Scratch

Before learning variables or writing code, you need to understand what PL/SQL actually does and why it exists.

1.1 What is a database?

Imagine a company that has hundreds of employees. The company needs to store information such as:

- Employee ID
- Employee name
- Salary
- Department
- Joining date

A database stores this information in an organized format. For example, an `EMPLOYEES` table might look like this:

EMPLOYEE_ID	NAME	SALARY	DEPARTMENT_ID
101	SMITH	3000	10
102	ALLEN	4000	20
103	JONES	5000	10
104	MARTIN	3500	30

Think of a table as a spreadsheet:

- Each **row** represents one employee.
- Each **column** represents a particular property of an employee.
- Each **cell** contains a value.

1.2 What is SQL?

SQL stands for **S**tructured **Q**uery **L**anguage. It allows you to communicate with a database - one statement at a time (`SELECT`, `INSERT`, `UPDATE`, `DELETE`...). For example, suppose you want to retrieve employees earning more than 3,500:

```
SELECT *
FROM employees
WHERE salary > 3500;
```

The database returns:

EMPLOYEE_ID	NAME	SALARY	DEPARTMENT_ID
102	ALLEN	4000	20
103	JONES	5000	10

Code	Meaning
`SELECT *`	Retrieve all columns
`FROM employees`	Read data from the employees table
`WHERE salary > 3500`	Keep only rows satisfying the condition

SQL is excellent for retrieving and manipulating database data. However, sometimes you need more complex logic. For example:

1. Retrieve an employee's salary.
2. Check whether the salary is below 4,000.
3. If it is, increase it by 10%.
4. Display the new salary.
5. Handle any errors.

You could execute multiple SQL statements separately, but PL/SQL allows you to combine SQL statements with programming logic.

1.3 What is PL/SQL?

> **DEFINITION** **PL/SQL** = **P**rocedural **L**anguage extension to **SQL**. It lets you add logic (`IF`, `LOOP`, variables) **around** your SQL, and sends a whole *block* of statements to the database engine at once. It is Oracle's procedural programming language for working with databases.

It combines SQL with programming features such as:

- Variables
- Constants
- Conditions
- Loops
- Procedures
- Functions
- Error handling
- Records and collections

SQL versus PL/SQL

Feature	SQL	PL/SQL
Nature	Declarative (what to do)	Procedural (how to do it)
Main purpose	Query and manipulate data	Combine SQL with procedural logic
Execution	Statement by statement	Block by block
Variables	Limited, depending on the environment	Built-in support for variables
Conditions / loops	Not ordinary procedural loops; conditions mainly through query expressions	`IF`, `CASE`, `LOOP`, `WHILE`, `FOR`
Custom errors	Database errors and SQL diagnostics only	Dedicated `EXCEPTION` section; can raise custom errors

Feature	SQL	PL/SQL
Reusable programs	Views and SQL routines, depending on the database	Procedures, functions and packages
Multiple statements	Usually submitted as individual SQL statements	Grouped into executable blocks
Used inside	Standalone	Can call SQL inside it

A simple visual representation

Imagine you need to increase an employee's salary by 10%. Using separate SQL operations you first run:

```
SELECT salary
FROM employees
WHERE employee_id = 101;
```

You examine the result and then execute another statement:

```
UPDATE employees
SET salary = salary * 1.10
WHERE employee_id = 101;
```

Using PL/SQL, you group statements into a block that Oracle executes as a unit:

```
BEGIN
    UPDATE employees
    SET salary = salary * 1.10
    WHERE employee_id = 101;
END;
/
```

Why it matters - performance visualization

ROUND TRIPS

```
Without PL/SQL (SQL only):                With PL/SQL (Block):
App --SQL1--> DB                          App --[SQL1 + SQL2 + SQL3 + Logic]--> DB
App <-------- DB                          App <---------------------------------- DB
App --SQL2--> DB         (3 network trips)                (1 network trip)
App <-------- DB
App --SQL3--> DB
App <-------- DB
```

> **CORRECTION** PL/SQL does **not** necessarily make every query faster. Its benefits include combining procedural operations, reducing application-to-database round trips in appropriate situations, and keeping database logic close to the data. PL/SQL and SQL also have separate execution engines that communicate when needed (this matters again in Chapter 22 - Bulk Binding).

Tools to write and run PL/SQL

- Oracle **SQL Developer** (graphical tool)
- **SQL*Plus** (command line)

Sample data used in this tutorial (optional)

If you want to follow along with the `employees` examples, you can create a small practice table:

```
CREATE TABLE employees (
    employee_id   NUMBER(6) PRIMARY KEY,
    first_name    VARCHAR2(50),
```

```
    salary        NUMBER(8,2),
    department_id NUMBER(4),
    hire_date     DATE
);

INSERT INTO employees VALUES (100, 'John',   5000, 10, DATE '2020-03-01');
INSERT INTO employees VALUES (101, 'Smith',  3000, 10, DATE '2021-07-15');
INSERT INTO employees VALUES (102, 'Allen',  4000, 20, DATE '2022-01-10');
COMMIT;
```

CHAPTER 2

The Structure of a PL/SQL Block

This is one of the most important concepts in the entire tutorial. Almost everything you learn later depends on understanding PL/SQL blocks. Every single PL/SQL program - no matter how advanced - is built from this one template.

2.1 What is a block?

A block is a logical unit of PL/SQL code. Think of it as a container holding related instructions. For example, a block might contain instructions to:

- Create a variable.
- Store a value in that variable.
- Calculate a result.
- Display the result.
- Handle an error.

A standard PL/SQL block has this structure:

```
DECLARE
   -- OPTIONAL: declare variables, constants, cursors, types
BEGIN
   -- MANDATORY: your actual logic and SQL statements go here
EXCEPTION
   -- OPTIONAL: handle errors gracefully
END;
/
```

VISUALIZATION OF A BLOCK

```
+-------------------------------+
| DECLARE   (Optional)          |  <- storage room (variables / constants)
+-------------------------------+
| BEGIN     (Mandatory)         |  <- the actual work happens here
|   ... statements ...          |
+-------------------------------+
| EXCEPTION (Optional)          |  <- safety net if something breaks
+-------------------------------+
| END;                          |
+-------------------------------+
/     <- tells SQL*Plus / SQL Developer: "run this block now"
```

KEY IDEA Only `BEGIN ... END;` is mandatory. `DECLARE` and `EXCEPTION` are optional sections.

This type of block (no name) is called an **anonymous block** - it runs once and is forgotten. Procedures and functions (Chapter 17) are named blocks that are saved permanently in the database.

2.2 The DECLARE section

This section is used to declare variables, constants, cursors, and other objects that belong to the block. Example:

```
DECLARE
    v_salary NUMBER;
    v_name   VARCHAR2(50);
```

Here, we declare two variables: `v_salary` can store a numeric value and `v_name` can store text. However, the declarations alone are not yet a complete executable block - the executable section must follow.

> **NOTE** `DECLARE` is optional. You can omit it when you do not need to declare anything.

2.3 The BEGIN section

The `BEGIN` section contains executable statements. Example:

```
DECLARE
    v_salary NUMBER;
BEGIN
    v_salary := 5000;
    DBMS_OUTPUT.PUT_LINE(v_salary);
END;
/
```

EXECUTION FLOW

```
          START
            |
            v
   Declare v_salary
            |
            v
   Assign value 5000
            |
            v
    Display the value
            |
            v
           END
```

OUTPUT

```
5000
```

The statement `v_salary := 5000;` assigns the number 5000 to the variable. The statement `DBMS_OUTPUT.PUT_LINE(v_salary);` displays its value.

> **TIP** If nothing is printed in SQL*Plus, run `SET SERVEROUTPUT ON` first. In SQL Developer, open the **DBMS Output** panel and enable it for your connection.

2.4 The EXCEPTION section

The `EXCEPTION` section handles errors that occur while executing the block. For example:

```
DECLARE
    v_result NUMBER;
BEGIN
    v_result := 10 / 0;
    DBMS_OUTPUT.PUT_LINE(v_result);
EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Cannot divide by zero.');
END;
/
```

OUTPUT

```
Cannot divide by zero.
```

Division by zero raises an Oracle exception. The handler catches that exception and displays a message. The `EXCEPTION` section is optional. (Full details in Chapter 15.)

2.5 The END statement and the slash

`END;` marks the end of the PL/SQL block. The semicolon is part of the PL/SQL syntax. The slash on the next line is different:

```
/
```

> **WARNING** The forward slash `/` is **NOT** part of PL/SQL syntax. In tools such as SQL*Plus and SQL Developer's script execution mode, `/` tells the client to submit the PL/SQL block currently in its buffer for execution ("execute everything above now"). It is not a PL/SQL statement and is not part of the block itself. The exact behaviour depends on the tool and execution mode.

2.6 Your first complete PL/SQL program

```
DECLARE
    v_name VARCHAR2(30);
    v_age  NUMBER;
BEGIN
    v_name := 'Tamzid';
    v_age  := 22;
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Age: '  || v_age);
END;
/
```

Here is what happens:

1. Oracle declares two variables.
2. It stores `'Tamzid'` in `v_name`.
3. It stores `22` in `v_age`.
4. It displays the name.
5. It displays the age.

ILLUSTRATIVE OUTPUT

```
Name: Tamzid
Age: 22
```

The age is just an example value.

Understanding ||

The operator `||` joins strings and other values converted to text. For example, if `v_name := 'Tamzid';` then `'Name: ' || 'Tamzid'` produces `Name: Tamzid`. This operation is called **concatenation**. It is similar to string concatenation in Python:

PYTHON EQUIVALENT

```
name = "Tamzid"
print("Name: " + name)
```

In PL/SQL, `||` is the usual string-concatenation operator.

CHAPTER 3

Placeholders

> **DEFINITION** **Placeholders** are **temporary storage areas** that Oracle creates to hold data temporarily so you can manipulate it while a PL/SQL block runs.

In PL/SQL, placeholders can be any of:

1. **Variables** - values that can change.
2. **Constants** - values that cannot change.
3. **Records** - groups of related values (Chapter 13).

To define a placeholder, you need:

- A **name**
- A **datatype**

Imagine a small box with a label. The box is named `v_salary`, and it contains the value 5000:

A PLACEHOLDER AS A LABELLED BOX



If you change the value, the same variable can hold a different number:

```
v_salary := 5000;
v_salary := 6000;
```

After the second assignment, `v_salary` contains 6000.

> **KEY IDEA** A **variable** is a named storage location. A **constant** is a named value that cannot be reassigned. A **record** groups multiple related fields together.

3.1 Common datatypes for placeholders

A data type determines what kind of value a placeholder can hold. The five essentials:

Datatype	Meaning	Example
`NUMBER(n,m)`	Numeric value; n = total digits, m = decimal digits	`NUMBER(6,2)` stores 1234.56
`CHAR(n)`	Fixed-length text	`CHAR(5)`
`VARCHAR2(n)`	Variable-length text (most used)	`VARCHAR2(20)`
`DATE`	Date and time	`DATE`
`BOOLEAN`	TRUE / FALSE / NULL	`BOOLEAN`

Each of these types is explained in detail in section 4.4.

CHAPTER 4

Variables

> **DEFINITION** **Variables** are placeholders that **store values that can change** during execution of a PL/SQL block.

4.1 Declaration syntax

```
variable_name datatype [NOT NULL] [:= initial_value];
```

Component	Meaning
`variable_name`	Name you choose for the variable (the identifier)
`datatype`	Any valid PL/SQL datatype - the type of value it can store
`NOT NULL`	*Optional.* Forces the variable to always have a value
`:=`	The assignment operator
`initial_value`	*Optional.* The default/initial value assigned during declaration

The square brackets indicate optional components. **Do not type the brackets themselves.** Each declaration is a separate statement and must end with `;`.

Basic example

```
DECLARE
   salary number(6);   -- "salary" is a variable, type NUMBER, length 6
BEGIN
   salary := 50000;
   dbms_output.put_line(salary);
END;
/
```

The same idea with the `v_` prefix convention:

```
DECLARE
    v_salary NUMBER(6);
BEGIN
    v_salary := 5000;
    DBMS_OUTPUT.PUT_LINE(v_salary);
END;
/
```

Here `v_salary NUMBER(6);` declares a numeric variable and `v_salary := 5000;` assigns a value to it.

4.2 Why use v_salary instead of salary?

You can name a variable `salary`. However, many developers use prefixes such as `v_` for local variables, for example `v_salary`, `v_name`, `v_age`, `v_total`. This is a **naming convention, not a requirement**. It makes code easier to read, especially when a variable has the same name as a database column:

```
DECLARE
    v_salary NUMBER;
BEGIN
    SELECT salary
    INTO   v_salary
```

```
    FROM   employees
    WHERE  employee_id = 101;
END;
/
```

The column is `salary`, while the PL/SQL variable is `v_salary`. The difference is immediately visible.

4.3 NOT NULL variables

> **WARNING** If a variable is declared `NOT NULL`, you **must** initialize it immediately - otherwise Oracle throws an error.

```
DECLARE
   salary number(4);
   dept   varchar2(10) NOT NULL := 'HR Dept';  -- must have a value right away
BEGIN
   salary := 2000;        -- value can change later, in BEGIN or EXCEPTION section
   dept   := 'Finance';   -- allowed, since NOT NULL only blocks NULL, not reassignment
END;
/
```

The variable `dept` must always have a non-NULL value. Valid versus invalid:

```
v_dept VARCHAR2(10) NOT NULL := 'HR Dept';   -- valid
v_dept VARCHAR2(10) NOT NULL;                -- INVALID: must be initialized
v_dept := NULL;                              -- INVALID: NOT NULL variable cannot hold NULL
```

What is NULL?

`NULL` represents a **missing, unknown or unavailable** value. It is not the same as:

- Zero
- An empty string in every programming language
- The Boolean value `FALSE`

```
v_salary := 0;      -- the salary is explicitly zero
v_salary := NULL;   -- the salary value is unknown or absent
```

This distinction matters in database logic.

4.4 Essential PL/SQL data types in detail

A. NUMBER

Used for numeric values.

```
DECLARE
    v_age    NUMBER;
    v_salary NUMBER(8, 2);
BEGIN
    v_age    := 22;
    v_salary := 45000.75;
    DBMS_OUTPUT.PUT_LINE(v_age);
    DBMS_OUTPUT.PUT_LINE(v_salary);
END;
/
```

`NUMBER(8,2)` means: **Precision** - up to 8 total decimal digits; **Scale** - 2 digits after the decimal point. For example `45000.75` contains seven digits in total, including the two decimal digits. The theoretical

maximum positive value with precision 8 and scale 2 is `999999.99`. A value outside the permitted range can raise a numeric overflow error.

The mathematical idea: precision and scale

Suppose you store x = 12345.67. The number contains seven digits in total, so the precision is 7. It has two digits after the decimal point, so the scale is 2.

```
                                    x = 12345.67
                            Precision = 7       Scale = 2
                Declaration that can represent x:   v_amount NUMBER(7,2);
```

This is especially useful for salaries, financial transactions and monetary calculations.

> **NOTE** Oracle `NUMBER` supports decimal precision; it is not restricted to integers.

B. CHAR

Used for fixed-length character data.

```
DECLARE
    v_code CHAR(5);
BEGIN
    v_code := 'AB';
    DBMS_OUTPUT.PUT_LINE('[' || v_code || ']');
END;
/
```

Conceptually, a fixed-length value is padded with spaces when necessary. The output is approximately:

OUTPUT

```
[AB   ]
```

The brackets help reveal the padding. Use `CHAR` when fixed-width character data is genuinely useful.

C. VARCHAR2

Used for variable-length character data.

```
DECLARE
    v_name VARCHAR2(50);
BEGIN
    v_name := 'Tamzid';
    DBMS_OUTPUT.PUT_LINE(v_name);
END;
/
```

The variable can hold a string of up to 50 characters, subject to Oracle's character-length and database configuration rules. Unlike `CHAR`, `VARCHAR2` does **not** pad shorter strings to the declared maximum length. For most ordinary text variables, `VARCHAR2` is a sensible default.

D. DATE

Used for dates and times.

```
DECLARE
    v_joining_date DATE;
BEGIN
    v_joining_date := DATE '2026-01-15';
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(v_joining_date, 'DD-MON-YYYY'));
END;
/
```

OUTPUT

```
15-JAN-2026
```

Oracle's DATE data type stores year, month, day, hour, minute and second. It does **not** store fractional seconds or a time-zone component. For those, Oracle provides TIMESTAMP and TIMESTAMP WITH TIME ZONE.

E. BOOLEAN

Used for logical values. A PL/SQL Boolean variable can represent TRUE, FALSE, or NULL (unknown).

```
DECLARE
    v_is_eligible BOOLEAN;
BEGIN
    v_is_eligible := TRUE;
    IF v_is_eligible THEN
        DBMS_OUTPUT.PUT_LINE('Eligible');
    END IF;
END;
/
```

OUTPUT

```
Eligible
```

> **WARNING** Oracle SQL and PL/SQL have different type capabilities, and SQL Boolean support depends on the Oracle Database version. Do not assume that every SQL column can use the same Boolean type as a PL/SQL variable.

Quick comparison

Data type	Example	Purpose
NUMBER	45000.75	Numbers and calculations
CHAR(5)	'A12'	Fixed-length character data
VARCHAR2(50)	'Tamzid'	Variable-length text
DATE	DATE '2026-01-15'	Date and time
BOOLEAN	TRUE	Logical conditions

4.5 Assigning values to variables

There are **two ways** to assign values to variables.

Method 1 - Direct assignment

```
variable_name := value;
```

In PL/SQL, v_salary := 5000; means "put the value 5000 into the variable named v_salary". The operator is :=. It is not the same as the equality operator used in conditions:

```
v_salary := 5000;          -- assignment: stores a value
IF v_salary = 5000 THEN    -- comparison: checks whether the value equals 5000
```

Python comparison

```
# Python
salary = 5000

-- PL/SQL
v_salary := 5000;
```

Python uses = for assignment. PL/SQL uses := for assignment and = for equality comparisons. This is an important difference to remember.

Updating a variable

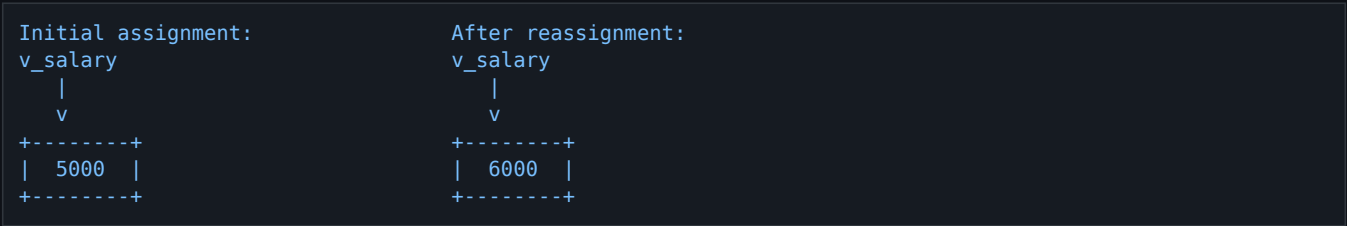
```
DECLARE
    v_salary NUMBER;
BEGIN
    v_salary := 5000;
    DBMS_OUTPUT.PUT_LINE('Old salary: ' || v_salary);
    v_salary := 6000;
    DBMS_OUTPUT.PUT_LINE('New salary: ' || v_salary);
END;
/
```

OUTPUT

```
Old salary: 5000
New salary: 6000
```

A variable can be reassigned repeatedly during execution, provided the new value is compatible with its data type and constraints.

VISUALIZATION



The variable’s name remains unchanged; the stored value changes.

Method 2 - From a database table using SELECT INTO

```
SELECT column_name INTO variable_name
FROM   table_name
[WHERE condition];
```

This is one of the most important operations in PL/SQL. Suppose the EMPLOYEES table contains:

EMPLOYEE_ID	NAME	SALARY
101	SMITH	3000
102	ALLEN	4000
103	JONES	5000

You want to retrieve Smith’s salary and store it in a variable:

```
DECLARE
    v_salary NUMBER;
BEGIN
    SELECT salary
    INTO   v_salary
    FROM   employees
```

```
    WHERE  employee_id = 101;
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
END;
/
```

OUTPUT

```
Salary: 3000
```

The complete PL/SQL statement does two things: (1) executes the query, (2) assigns the returned value to `v_salary`.

VISUALIZATION

```
EMPLOYEES TABLE
       |
       | WHERE employee_id = 101
       v
+------------------------+
| NAME:   SMITH          |
| SALARY: 3000           |
+------------------------+
       |
       | SELECT salary
       v
     3000
       |
       | INTO v_salary
       v
+------------------------+
| v_salary = 3000        |
+------------------------+
```

COMMON ERROR A normal `SELECT ... INTO` must return **exactly one row**. If it returns **no rows**, Oracle raises `NO_DATA_FOUND`. If it returns **more than one row**, Oracle raises `TOO_MANY_ROWS` (both handled in Chapter 15 - Exceptions).

```
SELECT salary
INTO   v_salary
FROM   employees
WHERE  department_id = 10;   -- may return several rows -> cannot fit in ONE scalar variable
```

If several employees belong to department 10, the query cannot assign all their salaries to a single scalar variable. You will learn how to handle multiple rows using records, cursors and collections later.

4.6 Full example from the lecture

```
DECLARE
   name1 varchar2(20);
   name2 varchar2(20);
BEGIN
   SELECT ename INTO name1 FROM emp WHERE sal = 800;
   SELECT ename INTO name2 FROM emp WHERE sal = 1600;
   dbms_output.put_line(name1 || ' ' || name2);
END;
/
```

What happens, step by step:

STEP BY STEP

```
Step 1: Declare two empty text boxes          -> name1 [ ]   name2 [ ]
Step 2: Query emp where sal = 800,  store it  -> name1 = "SMITH"
Step 3: Query emp where sal = 1600, store it  -> name2 = "ALLEN"
Step 4: Print "SMITH ALLEN"
```

CORRECTION The logic is valid if each query returns exactly one row, but there are two considerations. **First:** a salary condition does not guarantee a unique employee - if several employees earn 800, the first query raises `TOO_MANY_ROWS`. **Second:** the query selects employees by salary, not by name; the description says it retrieves SMITH and ALLEN, but the code does not actually enforce that.

If your intention is to retrieve SMITH and ALLEN, write:

```
DECLARE
    v_name1 VARCHAR2(20);
    v_name2 VARCHAR2(20);
BEGIN
    SELECT ename INTO v_name1 FROM emp WHERE ename = 'SMITH';
    SELECT ename INTO v_name2 FROM emp WHERE ename = 'ALLEN';
    DBMS_OUTPUT.PUT_LINE(v_name1 || ' ' || v_name2);
END;
/
```

OUTPUT (ASSUMING BOTH EMPLOYEES EXIST EXACTLY ONCE)

```
SMITH ALLEN
```

This example assumes the traditional Oracle SCOTT sample schema, which includes the `EMP` table. If you only need the names, you could also use one query with a cursor or a different SQL approach; for a beginner, two separate `SELECT INTO` statements make the assignment mechanism easier to understand.

CHAPTER 5

The %TYPE Attribute

PRO ADDITION Instead of hard-coding a datatype (risky - what if the column changes later?), you can **anchor** your variable to a column's type with `%TYPE`.

5.1 The problem with manually choosing data types

Suppose your database contains `employees.salary NUMBER(8,2)`. You could declare `v_salary NUMBER(8,2);` and this works. But imagine the schema later changes to `salary NUMBER(10,2)`. Your variable declaration no longer reflects the updated column definition. To avoid manually duplicating a column's data type, use `%TYPE`.

5.2 How %TYPE works

```
DECLARE
    v_salary employees.salary%TYPE;   -- automatically matches the column's datatype
BEGIN
    SELECT salary
    INTO   v_salary
    FROM   employees
    WHERE  employee_id = 101;
    DBMS_OUTPUT.PUT_LINE(v_salary);
END;
/
```

The declaration `v_salary employees.salary%TYPE;` means: give `v_salary` the data type associated with the `salary` column of the `employees` table.

VISUALIZATION

```
EMPLOYEES TABLE
      |
      v
salary column
NUMBER(8,2)
      |
      | %TYPE
      v
v_salary
NUMBER(8,2)
```

TIP If the DBA changes `salary` from `NUMBER(6,2)` to `NUMBER(8,2)`, your code adjusts - no need to edit the declaration. Recompiling dependent PL/SQL code generally allows the declaration to reflect the updated type.

WARNING The variable does **not** become a live database column, and `%TYPE` does **not** automatically enforce the column's `NOT NULL` constraint on the variable.

5.3 Why professionals use %TYPE

TABLE COLUMNS

```
EMPLOYEES
+-- EMPLOYEE_ID
+-- FIRST_NAME
+-- SALARY
+-- HIRE_DATE
```

Instead of manually defining each corresponding variable:

```
v_employee_id NUMBER(6);
v_first_name  VARCHAR2(50);
v_salary      NUMBER(8,2);
v_hire_date   DATE;
```

you can write:

```
v_employee_id employees.employee_id%TYPE;
v_first_name  employees.first_name%TYPE;
v_salary      employees.salary%TYPE;
v_hire_date   employees.hire_date%TYPE;
```

KEY IDEA **Professional rule:** when a variable is intended to hold a value from a particular database column, consider using %TYPE. This improves maintainability. (%ROWTYPE, the row-level cousin, is covered in Chapter 13.)

CHAPTER 6

PL/SQL Constants

> **DEFINITION** A **constant** is a **user-defined literal value** that stays unchanged throughout the program.

6.1 Why use constants?

Example: increasing salary by 25% across a huge program. Instead of writing `1.25` everywhere, declare it once as a constant. Next time you need 30%, change it in **one place**. A variable can be reassigned (`v_salary := 5000; v_salary := 6000;`), but a constant cannot be reassigned after initialization.

Suppose your company applies a standard 10% salary increase. Instead of writing `0.10` repeatedly, define a constant:

```
DECLARE
    c_increase_rate CONSTANT NUMBER := 0.10;
    v_salary        NUMBER := 5000;
BEGIN
    v_salary := v_salary + (v_salary * c_increase_rate);
    DBMS_OUTPUT.PUT_LINE('New salary: ' || v_salary);
END;
/
```

OUTPUT

```
New salary: 5500
```

The mathematics

```
                    Increase   = Salary x Rate
                             = 5000 x 0.10 = 500

                New Salary = Old Salary + Increase
                           = 5000 + 500  = 5500

                        Equivalent form:
        New Salary = Old Salary x (1 + Rate) = 5000 x 1.10 = 5500
```

VISUALIZATION

```
Original salary: 5000
        |
        v
Increase rate: 10%
        |
        v
Increase amount: 500
        |
        v
New salary: 5500
```

6.2 Constant declaration syntax

```
constant_name CONSTANT datatype := VALUE;
```

Part	Meaning
constant_name	Like a variable name
CONSTANT	Reserved keyword - locks the value
VALUE	**Mandatory** at declaration time - cannot be set later

```
DECLARE
   salary_increase CONSTANT number(3) := 10;
BEGIN
   dbms_output.put_line(salary_increase);
END;
/
```

ANOTHER EXAMPLE

```
DECLARE
    c_tax_rate CONSTANT NUMBER := 0.15;
BEGIN
    DBMS_OUTPUT.PUT_LINE(c_tax_rate);
END;
/
```

6.3 What NOT to do

ERROR EXAMPLE

```
DECLARE
   salary_increase CONSTANT number(3);     -- ERROR: no value given here
BEGIN
   salary_increase := 100;                 -- ERROR: cannot assign to a constant
   dbms_output.put_line(salary_increase);
END;
/
```

COMMON ERROR A constant **MUST** get its value at the exact moment of declaration. Never later. The constant above is missing its required initial value.

What happens if you change a constant that was properly initialized?

```
DECLARE
    c_tax_rate CONSTANT NUMBER := 0.15;
BEGIN
    c_tax_rate := 0.20;      -- Oracle rejects this assignment
END;
/
```

Oracle rejects the assignment because the constant cannot be reassigned.

Variable versus constant

Feature	Variable	Constant
Stores a value	Yes	Yes
Can be reassigned	Yes	No

Feature	Variable	Constant
Initialization required	Not always	Yes
Keyword	None required	`CONSTANT`
Example	`v_salary NUMBER := 5000;`	`c_rate CONSTANT NUMBER := 0.10;`

TIP A naming convention such as `c_` for constants helps distinguish them from variables.

CHAPTER 7

PL/SQL Literals

> **DEFINITION** A **literal** is an explicit value (not stored under a variable name) - like typing the number directly into the source code.

For example, in `v_age := 22;` the number `22` is a literal. Likewise in `v_name := 'Tamzid';` the text `'Tamzid'` is a literal. A literal is different from a variable: here `v_age` is an **identifier** (it names a program element) whereas `22` is a **numeric literal** (it represents a value).

Type	Examples
Numeric literal	`786`, `3.14`, `-500`
Character literal	`'A'`, `'9'`, `'$'`
String literal	`'adbms'`, `'Hello World'`
Boolean literal	`TRUE`, `FALSE`, `NULL`
Date / Time literal	`DATE '2025-01-01'`, `TIMESTAMP '2025-01-01 10:00:00'`

```
DECLARE
   a NUMBER := 786;                -- numeric literal
   b CHAR(1) := 'Y';               -- character literal
   c VARCHAR2(10) := 'adbms';      -- string literal
   d BOOLEAN := TRUE;              -- boolean literal
   e DATE := DATE '2025-06-15';    -- date literal
BEGIN
   NULL; -- do nothing, just a placeholder statement
END;
/
```

7.1 Numeric literals

```
DECLARE
    v_integer  NUMBER := 100;
    v_decimal  NUMBER := 45.75;
    v_negative NUMBER := -20;
BEGIN
    DBMS_OUTPUT.PUT_LINE(v_integer);
    DBMS_OUTPUT.PUT_LINE(v_decimal);
    DBMS_OUTPUT.PUT_LINE(v_negative);
END;
/
```

Numeric literals represent numbers. Examples include `100`, `45.75`, `-20` and `0.05`.

7.2 Character and string literals

Text literals are enclosed in **single** quotation marks:

```
v_name       := 'Tamzid';
v_department := 'Computer Science';
```

A text literal containing an apostrophe requires special handling - double the quote:

```
v_message := 'It''s a database tutorial';
```

OUTPUT

```
It's a database tutorial
```

The doubled single quotation mark represents one apostrophe inside the string. Character literals and string literals are not separate storage types in the same way that NUMBER and DATE are: in ordinary PL/SQL source code, both are represented using quoted character text.

7.3 Boolean literals

```
DECLARE
    v_passed BOOLEAN := TRUE;
BEGIN
    IF v_passed THEN
        DBMS_OUTPUT.PUT_LINE('Passed');
    END IF;
END;
/
```

The Boolean literals are TRUE and FALSE. Remember that a Boolean variable can also be NULL, representing an unknown logical state.

7.4 The NULL literal

```
DECLARE
    v_salary NUMBER;
BEGIN
    v_salary := NULL;
    IF v_salary IS NULL THEN
        DBMS_OUTPUT.PUT_LINE('Salary is unknown');
    END IF;
END;
/
```

OUTPUT

```
Salary is unknown
```

COMMON ERROR Never test for null with `v_salary = NULL`. In SQL and PL/SQL, use `IS NULL` or `IS NOT NULL`.

7.5 Date and time literals

Oracle supports ANSI date literals such as `DATE '2026-10-10'`.

```
DECLARE
    v_date DATE := DATE '2026-10-10';
BEGIN
    DBMS_OUTPUT.PUT_LINE(TO_CHAR(v_date, 'DD-MON-YYYY'));
END;
/
```

Using explicit date literals helps avoid ambiguity caused by session-specific date formats. For more advanced date/time work, Oracle provides TIMESTAMP and time-zone-aware timestamp types.

7.6 Are literals case-sensitive?

> **CORRECTION** The lecture states "PL/SQL literals are case-sensitive". That is true for the **contents of character literals**, but it needs a distinction from other elements of the language:

Element	Case-sensitive?	Example
Character data (inside quotes)	**Yes**	`'Hello'` and `'hello'` are different strings
Identifiers (unquoted)	No	`v_salary` and `V_SALARY` are the same identifier
Keywords	No	`BEGIN` and `begin` are equivalent
Boolean keywords	No	`TRUE` and `true` are equivalent

```
DECLARE
    v_name VARCHAR2(20) := 'Smith';
BEGIN
    DBMS_OUTPUT.PUT_LINE(v_name);
END;
/
```

The stored text retains the capitalization used in the literal.

CHAPTER 8

Scope of PL/SQL Variables

This topic is especially important when you begin writing larger programs. PL/SQL allows **nesting of blocks** - a block inside another block. This affects which parts of the code can "see" which variables.

8.1 What is scope?

> **DEFINITION** **Scope** describes where a variable can be accessed in a program.

```
DECLARE
    v_number NUMBER := 100;
BEGIN
    DBMS_OUTPUT.PUT_LINE(v_number);
END;
/
```

The variable `v_number` is declared in the outer block, so it can be accessed within that block's executable section. Now imagine placing another block inside the outer block.

8.2 Nested blocks

```
DECLARE
    v_num1 NUMBER := 100;
    v_num2 NUMBER := 200;
BEGIN
    DECLARE
        v_mult NUMBER;
    BEGIN
        v_mult := v_num1 * v_num2;
        DBMS_OUTPUT.PUT_LINE(v_mult);
    END;
END;
/
```

OUTPUT

```
20000
```

The inner block can access `v_num1` and `v_num2` because those variables were declared in the outer block. However, `v_mult` is declared inside the inner block and is not accessible after that inner block ends.

VISUALIZATION

```
DECLARE
   var_num1 NUMBER;      +
   var_num2 NUMBER;      |  OUTER BLOCK (global here)
BEGIN                    |
   var_num1 := 100;      |  accessible EVERYWHERE (outer + inner)
   var_num2 := 200;      |
                         |
   DECLARE               +
      var_mult NUMBER;   |  INNER BLOCK (local here)
   BEGIN                 |  accessible ONLY inside this inner block
      var_mult := var_num1 * var_num2;   -- OK: can use outer vars here
   END;                  +
                         |
   -- var_mult := 500;   -- ERROR! Not visible outside the inner block
END;                     +
/
```

THE ROOM-IN-A-ROOM PICTURE



Think of the inner block as a room inside a larger room. Someone in the inner room can access information available in the outer room, but an object that exists only in the inner room does not automatically become available outside it.

Example from the slides

```
DECLARE
   var_num1 number;
   var_num2 number;
BEGIN
   var_num1 := 100;
   var_num2 := 200;
   DECLARE
      var_mult number;
   BEGIN
      var_mult := var_num1 * var_num2;  -- = 20000
   END;
END;
/
```

KEY IDEA `var_mult` **cannot** be referenced after the inner `END;` - it "dies" there.

8.3 Local, outer-block and global variables

Type	Declared in	Accessible from
Local variable	Inner block	Only within that inner block
Global variable (lecture term)	Outer block	The outer block AND all nested inner blocks

CORRECTION "Local" and "global" is a useful introductory distinction, but professionals use more precise terms. **Local variable:** declared within a particular block or subprogram. **Outer-block variable:** declared in an enclosing block. **Global variable:** usually refers to a value accessible across a broader scope, such as a **package variable** (Chapter 18). An outer-block variable is not necessarily globally accessible throughout the database.

```
DECLARE
    v_outer NUMBER := 10;
BEGIN
    DECLARE
        v_inner NUMBER := 20;
    BEGIN
        DBMS_OUTPUT.PUT_LINE(v_outer);
```

```
        DBMS_OUTPUT.PUT_LINE(v_inner);
    END;
    DBMS_OUTPUT.PUT_LINE(v_outer);
    -- v_inner is not accessible here.
END;
/
```

The outer variable is accessible both inside and outside the nested block, as long as execution remains within its scope. The inner variable is accessible only inside the inner block.

8.4 Variables with the same name (shadowing)

```
DECLARE
    v_value NUMBER := 100;
BEGIN
    DECLARE
        v_value NUMBER := 200;
    BEGIN
        DBMS_OUTPUT.PUT_LINE(v_value);
    END;
    DBMS_OUTPUT.PUT_LINE(v_value);
END;
/
```

OUTPUT

```
200
100
```

Inside the inner block, the locally declared `v_value` takes precedence. Outside the inner block, the outer `v_value` is used. This is called **variable shadowing**.

> **TIP** Although Oracle supports shadowing, using the same variable name in nested blocks makes code harder to understand. Descriptive and distinct names are generally preferable.

CHAPTER 9

SQL and Mathematical Operations in PL/SQL

Before moving into more advanced programming structures, let us connect variables to practical calculations.

9.1 Arithmetic operators

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
**	Exponentiation	2 ** 3	8

```
DECLARE
    v_a NUMBER := 10;
    v_b NUMBER := 5;
BEGIN
    DBMS_OUTPUT.PUT_LINE(v_a + v_b);
    DBMS_OUTPUT.PUT_LINE(v_a - v_b);
    DBMS_OUTPUT.PUT_LINE(v_a * v_b);
    DBMS_OUTPUT.PUT_LINE(v_a / v_b);
    DBMS_OUTPUT.PUT_LINE(v_a ** 3);
END;
/
```

OUTPUT

```
15
5
50
2
1000
```

The last result is 1000 because `v_a ** 3` means:

```
10^3 = 10 x 10 x 10 = 1000
```

9.2 Comparison operators

Operator	Meaning
=	Equal to
<>	Not equal to
!=	Not equal to
>	Greater than
<	Less than

Operator	Meaning
>=	Greater than or equal to
<=	Less than or equal to

```
DECLARE
    v_salary NUMBER := 5000;
BEGIN
    IF v_salary >= 4000 THEN
        DBMS_OUTPUT.PUT_LINE('Salary is sufficient');
    END IF;
END;
/
```

OUTPUT

```
Salary is sufficient
```

9.3 Logical operators

PL/SQL supports AND, OR and NOT.

```
DECLARE
    v_age    NUMBER := 25;
    v_salary NUMBER := 5000;
BEGIN
    IF v_age >= 18 AND v_salary >= 4000 THEN
        DBMS_OUTPUT.PUT_LINE('Conditions satisfied');
    END IF;
END;
/
```

Both conditions must be true for the AND expression to be true. Mathematically this resembles a logical conjunction:

```
    A AND B  (A /\ B)   both A and B must be true
A OR  B  (A \/ B)   at least one condition must be true
  NOT A    (~A)       the logical result is reversed
```

> **WARNING** SQL and PL/SQL also use **three-valued logic** involving NULL. When a condition depends on an unknown value, its result may be *unknown* (neither true nor false) rather than simply true or false.

Three-valued logic truth tables

A	B	A AND B	A OR B
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	NULL	NULL	TRUE
FALSE	FALSE	FALSE	FALSE
FALSE	NULL	FALSE	NULL
NULL	NULL	NULL	NULL

CHAPTER 10

Running SQL Inside PL/SQL (DML)

You can run **INSERT, UPDATE, DELETE and MERGE** directly inside the BEGIN section - this is where PL/SQL and SQL truly connect.

```
DECLARE
   v_salary employees.salary%TYPE;
BEGIN
   SELECT salary INTO v_salary FROM employees WHERE employee_id = 100;
   UPDATE employees SET salary = v_salary * 1.10 WHERE employee_id = 100;
   COMMIT;  -- save the change permanently (good practice)
END;
/
```

KEY IDEA **Rule of thumb:** SELECT needs INTO when used inside PL/SQL (to store the result somewhere). INSERT, UPDATE and DELETE do not need INTO.

10.1 Other DML examples

```
BEGIN
    INSERT INTO employees (employee_id, first_name, salary, department_id)
    VALUES (200, 'Maya', 4200, 20);

    DELETE FROM employees WHERE employee_id = 200;

    COMMIT;
END;
/
```

10.2 The implicit cursor: how many rows did my DML affect?

After every INSERT, UPDATE, DELETE or single-row SELECT INTO, Oracle fills in an implicit cursor called SQL. Its attributes tell you what happened (the same attribute names appear again for explicit cursors in Chapter 14).

```
BEGIN
    UPDATE employees SET salary = salary + 100 WHERE department_id = 10;
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' row(s) updated');
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('Nothing matched the WHERE clause');
    END IF;
    ROLLBACK;   -- undo this demo change
END;
/
```

PRO ADDITION Transactions (COMMIT, ROLLBACK, SAVEPOINT) are covered fully in Chapter 20.

Checkpoint: what you should understand so far

You have now covered every topic from the foundational part of the original lecture:

Topic from the material	Covered?	Main idea
PL/SQL placeholders	Yes	Storage for values and structured data

Topic from the material	Covered?	Main idea
Variables	Yes	Named storage that can be reassigned
Data types	Yes	Determine what values can be stored
`NOT NULL`	Yes	Prevents null values in a variable
Direct assignment	Yes	`v_salary := 5000;`
`SELECT INTO`	Yes	Retrieves a single row into PL/SQL variables
Variable scope	Yes	Determines where a variable is accessible
Nested blocks	Yes	Blocks can contain other blocks
Constants	Yes	Values cannot be reassigned
Numeric literals	Yes	Values such as 100 and 45.75
Character / string literals	Yes	Values such as 'Tamzid'
Boolean literals	Yes	TRUE and FALSE
`NULL`	Yes	Represents an absent or unknown value
Date / time literals	Yes	Explicitly represent dates and times
`%TYPE`	Yes	Anchors a variable's type to a database column
Mathematical operators	Added	Arithmetic, comparison and logical operations
SQL (DML) inside PL/SQL	Added	INSERT / UPDATE / DELETE in the BEGIN section

Practice exercise

Write a PL/SQL block that performs the following tasks:

1. Declare a variable named `v_salary`.
2. Initialize it to 4000.
3. Declare a constant named `c_increase_rate` with a value of 0.15.
4. Calculate the salary increase.
5. Store the new salary in another variable named `v_new_salary`.
6. Display the original salary, increase amount and new salary.

Use these equations:

```
Increase   = Salary x Rate
New Salary = Salary + Increase
```

Your expected results should be:

Calculation	Result
Original salary	4000
Increase rate	15%
Increase amount	600
New salary	4600

Sample solution (try first, then compare)

```
DECLARE
    v_salary        NUMBER := 4000;
    c_increase_rate CONSTANT NUMBER := 0.15;
    v_increase      NUMBER;
    v_new_salary    NUMBER;
BEGIN
    v_increase   := v_salary * c_increase_rate;
    v_new_salary := v_salary + v_increase;
    DBMS_OUTPUT.PUT_LINE('Original salary : ' || v_salary);
    DBMS_OUTPUT.PUT_LINE('Increase amount : ' || v_increase);
    DBMS_OUTPUT.PUT_LINE('New salary      : ' || v_new_salary);
END;
/
```

OUTPUT

```
Original salary : 4000
Increase amount : 600
New salary      : 4600
```

PART II

Intermediate

Control the flow of your programs, work with entire rows, process many rows with cursors, catch errors gracefully and keep lists of data in memory.

CHAPTER 11

Conditional Logic: IF and CASE

11.1 IF-THEN-ELSIF (complex conditions)

```
DECLARE
   v_marks NUMBER := 75;
BEGIN
   IF v_marks >= 90 THEN
      dbms_output.put_line('Grade A');
   ELSIF v_marks >= 70 THEN
      dbms_output.put_line('Grade B');
   ELSE
      dbms_output.put_line('Grade C');
   END IF;
END;
/
```

OUTPUT

```
Grade B
```

DECISION FLOW

```
          v_marks = 75
               |
        >= 90? --yes--> Grade A
               |no
        >= 70? --yes--> Grade B   <-- taken
               |no
               +--------> Grade C
```

> **WARNING** The keyword is ELSIF (no "E" between S and I), and every IF must be closed with END IF;.

IF-THEN and IF-THEN-ELSE

```
IF v_salary > 5000 THEN
    v_bonus := 500;
END IF;

IF v_salary > 5000 THEN
    v_bonus := 500;
ELSE
    v_bonus := 100;
END IF;
```

11.2 CASE (clean value matching)

```
DECLARE
   v_day NUMBER := 3;
BEGIN
   CASE v_day
      WHEN 1 THEN dbms_output.put_line('Monday');
      WHEN 2 THEN dbms_output.put_line('Tuesday');
      WHEN 3 THEN dbms_output.put_line('Wednesday');
      ELSE dbms_output.put_line('Other day');
   END CASE;
END;
/
```

OUTPUT

```
Wednesday
```

Searched CASE (conditions instead of fixed values)

```
DECLARE
    v_salary NUMBER := 4500;
    v_band   VARCHAR2(10);
BEGIN
    v_band := CASE
                  WHEN v_salary >= 5000 THEN 'HIGH'
                  WHEN v_salary >= 3000 THEN 'MEDIUM'
                  ELSE 'LOW'
              END;
    DBMS_OUTPUT.PUT_LINE(v_band);
END;
/
```

OUTPUT

```
MEDIUM
```

COMMON ERROR A `CASE` **statement** without an `ELSE` branch raises the `CASE_NOT_FOUND` exception if no `WHEN` matches. Add an `ELSE` unless you want that behaviour.

CHAPTER 12

Loops and Iteration

12.1 Basic LOOP (needs a manual exit)

```
DECLARE
   i NUMBER := 1;
BEGIN
   LOOP
      dbms_output.put_line(i);
      i := i + 1;
      EXIT WHEN i > 5;    -- stop condition: mandatory, else infinite loop!
   END LOOP;
END;
/
```

OUTPUT

```
1
2
3
4
5
```

12.2 WHILE LOOP (condition checked FIRST)

```
DECLARE
   i NUMBER := 1;
BEGIN
   WHILE i <= 5 LOOP
      dbms_output.put_line(i);
      i := i + 1;
   END LOOP;
END;
/
```

12.3 FOR LOOP (fixed number of iterations)

```
BEGIN
   FOR i IN 1..5 LOOP
      dbms_output.put_line(i);
   END LOOP;
END;
/
```

> **TIP** The loop counter `i` is declared **automatically** by the `FOR` loop and exists only inside it. You cannot assign to it. Add `REVERSE` to count down: `FOR i IN REVERSE 1..5 LOOP`.

12.4 CONTINUE statement (skip to the next iteration)

```
BEGIN
   FOR i IN 1..5 LOOP
      IF i = 3 THEN
         CONTINUE;  -- skips printing 3
      END IF;
      dbms_output.put_line(i);
   END LOOP;
END;
/
```

OUTPUT

```
1
2
4
5
```

12.5 Loop comparison

Loop type	Condition checked	Best used when
Basic LOOP	Manually via EXIT WHEN	Unknown number of iterations
WHILE	Before each iteration	Condition-based repetition
FOR	Automatic (range)	Known fixed number of iterations

12.6 Combined example: FOR LOOP + IF

```
BEGIN
   FOR i IN 1..10 LOOP
      IF MOD(i,2) = 0 THEN
         dbms_output.put_line(i || ' is Even');
      ELSE
         dbms_output.put_line(i || ' is Odd');
      END IF;
   END LOOP;
END;
/
```

OUTPUT

```
1 is Odd
2 is Even
3 is Odd
...
10 is Even
```

12.7 Nested loops and loop labels

```
BEGIN
    <<outer_loop>>
    FOR i IN 1..3 LOOP
        FOR j IN 1..3 LOOP
            EXIT outer_loop WHEN i * j > 4;   -- leaves BOTH loops
            DBMS_OUTPUT.PUT_LINE(i || ' x ' || j || ' = ' || i*j);
        END LOOP;
    END LOOP outer_loop;
END;
/
```

CHAPTER 13

Records: Managing Entire Rows

A `SELECT INTO` with single variables works fine for one column. But what if you want to grab an **entire row**? Use a **record**.

VISUALIZATION

```
Table: employees                Record: v_employee_row
+-----+-------+--------+        +----------------------+
| id  | name  | salary |  --->  | id     = 100         |
+-----+-------+--------+        | name   = 'John'      |
| 100 | John  | 5000   |        | salary = 5000        |
+-----+-------+--------+        +----------------------+
        (one row packed into ONE variable)
```

13.1 %ROWTYPE - auto-matches a table's structure

```
DECLARE
   v_employee_row employees%ROWTYPE;  -- auto creates a field for every column
BEGIN
   SELECT * INTO v_employee_row FROM employees WHERE employee_id = 100;
   dbms_output.put_line(v_employee_row.first_name || ' earns ' || v_employee_row.salary);
END;
/
```

13.2 User-defined records - custom structure

```
DECLARE
   TYPE t_custom_emp IS RECORD (
      emp_name   varchar2(50),
      department varchar2(50)
   );
   v_custom t_custom_emp;
BEGIN
   v_custom.emp_name   := 'Alice';
   v_custom.department := 'Finance';
   dbms_output.put_line(v_custom.emp_name || ' - ' || v_custom.department);
END;
/
```

OUTPUT

```
Alice - Finance
```

%ROWTYPE	User-defined record
Mirrors a real table exactly	Custom fields you design
Auto-adjusts if the table changes	Must manually update if needs change
Use `SELECT *` to fill it	Fill field-by-field or with a custom query

TIP You can also anchor a record to a cursor: `v_row c_emps%ROWTYPE;` (see Chapter 14). Fields are accessed with dot notation: `record_name.field_name`.

CHAPTER 14

Cursors: Handling Multi-Row Queries

> **WARNING** Recall: `SELECT INTO` **crashes** if it gets more than one row (`TOO_MANY_ROWS`). To process **many rows**, use a **cursor** - think of it as a "pointer" moving row by row over a result set.

VISUALIZATION

```
Query Result Set (memory):
+---------+--------+
| Alice   | 5000   | <- Row 1   (cursor points here first)
+---------+--------+
| Bob     | 6000   | <- Row 2
+---------+--------+
| Carl    | 7000   | <- Row 3
+---------+--------+

Cursor moves: OPEN -> FETCH (row1) -> FETCH (row2) -> FETCH (row3) -> CLOSE
```

14.1 Explicit cursor lifecycle (manual)

```
DECLARE
   CURSOR c_emps IS SELECT first_name, salary FROM employees WHERE department_id = 10;
   v_name   employees.first_name%TYPE;
   v_salary employees.salary%TYPE;
BEGIN
   OPEN c_emps;                              -- Step 1: open the cursor
   LOOP
      FETCH c_emps INTO v_name, v_salary;    -- Step 2: grab next row
      EXIT WHEN c_emps%NOTFOUND;             -- Step 3: stop when no more rows
      dbms_output.put_line(v_name || ' - ' || v_salary);
   END LOOP;
   CLOSE c_emps;                             -- Step 4: close cursor (free memory)
END;
/
```

14.2 Cursor attributes

Attribute	Meaning
`%FOUND`	TRUE if the last fetch returned a row
`%NOTFOUND`	TRUE if the last fetch did NOT return a row (used to exit loops)
`%ROWCOUNT`	Number of rows fetched so far
`%ISOPEN`	TRUE if the cursor is currently open

14.3 Cursor FOR loop (best practice - automatic)

```
DECLARE
   CURSOR c_emps IS SELECT first_name, salary FROM employees WHERE department_id = 10;
BEGIN
   FOR r_emp IN c_emps LOOP      -- auto OPEN, auto FETCH, auto CLOSE!
      dbms_output.put_line(r_emp.first_name || ' - ' || r_emp.salary);
   END LOOP;
END;
/
```

Explicit cursor (manual)	Cursor FOR loop (automatic)
You write OPEN / FETCH / CLOSE	Oracle does it for you
More control (e.g. multiple fetches)	Shorter, cleaner, safer (no forgot-to-close bugs)

14.4 Cursors with parameters

```
DECLARE
    CURSOR c_dept(p_dept NUMBER) IS
        SELECT first_name, salary FROM employees WHERE department_id = p_dept;
BEGIN
    FOR r IN c_dept(10) LOOP
        DBMS_OUTPUT.PUT_LINE(r.first_name || ' (dept 10)');
    END LOOP;
    FOR r IN c_dept(20) LOOP
        DBMS_OUTPUT.PUT_LINE(r.first_name || ' (dept 20)');
    END LOOP;
END;
/
```

14.5 Implicit versus explicit cursors

Kind	Created by	Used for
Implicit (`SQL%...`)	Oracle, automatically, for every DML and `SELECT INTO`	Checking `SQL%ROWCOUNT` after DML
Explicit	You, with `CURSOR ... IS` in `DECLARE`	Processing multi-row queries row by row

CHAPTER 15

Exception Handling: Catching Errors

DEFINITION **Goal:** database errors should not crash your whole application. The EXCEPTION block is your **safety net**.

STRUCTURE

```
BEGIN
   risky_statement();   -- error occurs here!
   -- rest of the BEGIN section is SKIPPED
EXCEPTION
   WHEN error_type THEN
      handle_it();      -- control jumps here instead
END;
```

15.1 Predefined exceptions (built into Oracle)

Exception name	Fires when
NO_DATA_FOUND	SELECT INTO returns 0 rows
TOO_MANY_ROWS	SELECT INTO returns more than 1 row
ZERO_DIVIDE	Division by zero
VALUE_ERROR	Data type conversion / size error
DUP_VAL_ON_INDEX	Unique constraint violation
INVALID_NUMBER	A string cannot be converted to a number in SQL
CASE_NOT_FOUND	No WHEN branch matched in a CASE statement without ELSE
CURSOR_ALREADY_OPEN	Opening a cursor that is already open
OTHERS	Catch-all handler for anything not handled above

```
DECLARE
    v_salary employees.salary%TYPE;
BEGIN
    SELECT salary INTO v_salary FROM employees WHERE employee_id = 9999;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee with that ID.');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one row returned.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/
```

TIP Inside a handler, SQLCODE returns the error number and SQLERRM returns the message text. Always put WHEN OTHERS **last**.

15.2 User-defined exceptions (your own rules)

```
DECLARE
   e_salary_too_low EXCEPTION;          -- 1. declare it
   v_sal NUMBER := 500;
BEGIN
   IF v_sal < 1000 THEN
      RAISE e_salary_too_low;           -- 2. raise it manually
   END IF;
EXCEPTION
   WHEN e_salary_too_low THEN           -- 3. handle it
      RAISE_APPLICATION_ERROR(-20001, 'Salary violates minimum wage policy.');
   WHEN NO_DATA_FOUND THEN
      dbms_output.put_line('No matching records found.');
END;
/
```

15.3 RAISE_APPLICATION_ERROR

- Used to **stop execution** and send a **custom error code + message** back to the user/application.
- The error code **must** be between **-20000 and -20999** (the range reserved for custom errors).

15.4 Error propagation

VISUALIZATION

```
 OUTER BLOCK
 +-----------------------------+
 | INNER BLOCK                 |
 | +-------------------------+ |
 | | error occurs!           | |
 | | no EXCEPTION handler    | |
 | +-----------+-------------+ |
 |             | bubbles up    |
 |     EXCEPTION <-------------+
 |     WHEN OTHERS THEN caught here!
 +-----------------------------+
```

If the inner block cannot handle an error, it "bubbles up" to the outer block's EXCEPTION section. If **no** block handles it, the error reaches the user/application.

15.5 Continuing after an error

```
BEGIN
    FOR r IN (SELECT employee_id FROM employees) LOOP
        BEGIN                                   -- inner block protects ONE iteration
            UPDATE employees SET salary = salary / (r.employee_id - 100)
            WHERE employee_id = r.employee_id;
        EXCEPTION
            WHEN ZERO_DIVIDE THEN
                DBMS_OUTPUT.PUT_LINE('Skipped employee ' || r.employee_id);
        END;
    END LOOP;
    ROLLBACK;
END;
/
```

CHAPTER 16

Collections: Arrays in PL/SQL

> **DEFINITION** **Collections** let you store **lists of data in memory** instead of repeatedly querying the database.

Type	Size	Indexed by	Stored in DB column?	Best for
Associative array (index-by table)	Unbounded, sparse	Number or string	No	Fast temporary lookups
Nested table	Unbounded	Number only	Yes	Large flexible lists stored in the DB
VARRAY	Fixed max size	Number only	Yes	Small, size-limited lists

VISUALIZATION

```
Associative Array:        Nested Table:         VARRAY (max 3):
 [1] -> Alice              [1] -> Alice           [1] -> Alice
 [2] -> Bob                [2] -> Bob             [2] -> Bob
 [5] -> Carl (gaps OK!)    [3] -> Carl            [3] -> Carl
                                                  [4] -> ERROR (exceeds limit)
```

16.1 Collection methods

Method	Purpose
`.COUNT`	Total number of elements
`.FIRST` / `.LAST`	Lowest / highest populated index
`.DELETE`	Remove element(s)
`.EXISTS(n)`	Check if index `n` has a value
`.NEXT(n)` / `.PRIOR(n)`	Next / previous populated index (safe for sparse collections)
`.EXTEND`	Add an empty element (nested tables and VARRAYs)

16.2 Associative array example

```
DECLARE
   TYPE t_names IS TABLE OF VARCHAR2(50) INDEX BY PLS_INTEGER;  -- Associative Array
   v_employees t_names;
BEGIN
   v_employees(1) := 'Alice';
   v_employees(2) := 'Bob';

   dbms_output.put_line('Total employees: ' || v_employees.COUNT);
   dbms_output.put_line('First employee: ' || v_employees(v_employees.FIRST));

   v_employees.DELETE(2);  -- Removes Bob
END;
/
```

OUTPUT

```
Total employees: 2
First employee: Alice
```

16.3 Nested table and VARRAY examples

```
DECLARE
    TYPE t_list  IS TABLE OF VARCHAR2(30);       -- nested table
    TYPE t_three IS VARRAY(3) OF VARCHAR2(30);   -- VARRAY, max 3 elements
    v_list  t_list  := t_list('Alice', 'Bob');   -- constructor initializes it
    v_three t_three := t_three('Red', 'Green');
BEGIN
    v_list.EXTEND;                -- make room for one more
    v_list(3) := 'Carl';
    FOR i IN 1..v_list.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE(v_list(i));
    END LOOP;

    v_three.EXTEND;
    v_three(3) := 'Blue';         -- OK: exactly at the limit
    -- v_three.EXTEND; would raise SUBSCRIPT_BEYOND_LIMIT
END;
/
```

COMMON ERROR Nested tables and VARRAYs must be **initialized** with their constructor (`t_list(...)`) before use, otherwise Oracle raises `COLLECTION_IS_NULL`. Associative arrays need no initialization.

PART III

Advanced

Package your logic into reusable stored code, automate reactions to data changes, control transactions and build SQL at run time.

CHAPTER 17

Procedures and Functions: Reusable Code

> **DEFINITION** Anonymous blocks run once and vanish. **Procedures and functions are saved permanently in the database** for reuse.

17.1 Parameter modes

Mode	Direction	Analogy
IN	Caller -> subprogram (read-only)	Sending a letter (you cannot change what was sent)
OUT	Subprogram -> caller	Receiving a package back
IN OUT	Both ways	Borrowing a book, writing in it, returning it modified

17.2 Procedure versus function

Procedure	Function
Performs an action	Computes and returns ONE value
Does not have to return anything	MUST return a value
Called as a standalone statement	Can be used inside a SELECT statement

17.3 Function example

```
CREATE OR REPLACE FUNCTION get_annual_salary(p_monthly_sal IN NUMBER)
RETURN NUMBER IS
BEGIN
   RETURN p_monthly_sal * 12;
END;
/

-- Usage:
SELECT get_annual_salary(5000) FROM dual;
```

OUTPUT

```
GET_ANNUAL_SALARY(5000)
-----------------------
                  60000
```

17.4 Procedure example

```
CREATE OR REPLACE PROCEDURE give_raise(p_emp_id IN NUMBER, p_amount IN NUMBER) IS
BEGIN
   UPDATE employees SET salary = salary + p_amount WHERE employee_id = p_emp_id;
END;
/

-- Usage:
BEGIN
```

```
   give_raise(100, 500);
END;
/
```

17.5 OUT and IN OUT parameters

```
CREATE OR REPLACE PROCEDURE get_emp_info(
    p_id     IN  NUMBER,
    p_name   OUT VARCHAR2,
    p_salary IN OUT NUMBER) IS
BEGIN
    SELECT first_name INTO p_name FROM employees WHERE employee_id = p_id;
    p_salary := p_salary * 1.10;            -- reads the incoming value, returns a new one
END;
/

DECLARE
    v_name   VARCHAR2(50);
    v_salary NUMBER := 1000;
BEGIN
    get_emp_info(100, v_name, v_salary);
    DBMS_OUTPUT.PUT_LINE(v_name || ' ' || v_salary);
END;
/
```

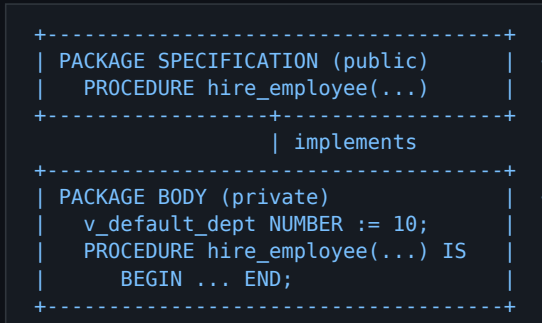
TIP Use **positional** (`give_raise(100, 500)`) or **named** notation (`give_raise(p_amount => 500, p_emp_id => 100)`). A parameter can have a default: `p_amount IN NUMBER DEFAULT 100`. Prefixing parameters with `p_` avoids clashes with column names.

CHAPTER 18

Packages: Organizing Code

DEFINITION Packages group related variables, cursors, procedures and functions into **one logical unit** - like a toolbox.

VISUALIZATION




```plsql
-- SPECIFICATION (public interface)
CREATE OR REPLACE PACKAGE hr_pkg IS
   PROCEDURE hire_employee(p_name IN VARCHAR2);
END hr_pkg;
/

-- BODY (private implementation)
CREATE OR REPLACE PACKAGE BODY hr_pkg IS
   v_default_dept NUMBER := 10;   -- hidden from the outside world

   PROCEDURE hire_employee(p_name IN VARCHAR2) IS
   BEGIN
      INSERT INTO employees (name, dept_id) VALUES (p_name, v_default_dept);
   END;
END hr_pkg;
/

-- Usage:
BEGIN
   hr_pkg.hire_employee('David');
END;
/
```

NOTE The example inserts into columns `name` and `dept_id`; adapt the column names to match your own table (the practice `employees` table in Chapter 1 uses `first_name` and `department_id`).

18.1 Extra concepts

- **Subprogram overloading:** same procedure name, different parameters (e.g. `find_employee(id)` vs `find_employee(name)`).
- **Session state:** package-level variables (like `v_default_dept`) keep their value for your **entire database session** - acting like "memory" between calls. This is also what people mean by a real *global variable* in PL/SQL.

OVERLOADING (SPECIFICATION ONLY)

```plsql
CREATE OR REPLACE PACKAGE finder IS
    FUNCTION find_employee(p_id   IN NUMBER)   RETURN VARCHAR2;   -- overload 1
    FUNCTION find_employee(p_name IN VARCHAR2) RETURN NUMBER;     -- overload 2
END finder;
```

```
/
```

CHAPTER 19

Database Triggers: Event-Driven Automation

> **DEFINITION** Triggers are code that **automatically fires** when an event (INSERT / UPDATE / DELETE) happens on a table.

TIMING AND LEVEL

```
                Event: UPDATE salary
                        |
          +-------------+-------------+
     BEFORE trigger            AFTER trigger
    (fires before change)     (fires after change)
          |                           |
   Statement-level: fires ONCE total (regardless of rows)
   Row-level (FOR EACH ROW): fires ONCE PER ROW affected
```

Concept	Meaning
Statement-level trigger	Fires exactly once per SQL statement
Row-level trigger (FOR EACH ROW)	Fires once per row affected
:NEW	The incoming (new) value being inserted / updated
:OLD	The previous value before the change
INSTEAD OF trigger	Used on views; replaces the DML action entirely
BEFORE / AFTER	Controls whether the trigger fires before or after the actual event

```
CREATE OR REPLACE TRIGGER trg_audit_salary
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
   IF :NEW.salary < :OLD.salary THEN
      INSERT INTO salary_audit (emp_id, old_sal, new_sal, change_date)
      VALUES (:NEW.employee_id, :OLD.salary, :NEW.salary, SYSDATE);
   END IF;
END;
/
```

This logs every time a salary is **decreased** - automatically, with no extra code needed in the application.

19.1 Pseudorecords :NEW and :OLD

Event	:OLD	:NEW
INSERT	Not available (all NULL)	The row being inserted
UPDATE	Row before the change	Row after the change
DELETE	The row being deleted	Not available (all NULL)

19.2 A BEFORE trigger that fixes data

```
CREATE OR REPLACE TRIGGER trg_upper_name
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    :NEW.first_name := UPPER(:NEW.first_name);   -- BEFORE row triggers may modify :NEW
END;
/
```

WARNING Triggers run invisibly. Keep them short and predictable; heavy logic inside triggers makes applications hard to debug.

CHAPTER 20

Transactions: COMMIT, ROLLBACK and SAVEPOINT

A **transaction** is a group of DML statements that must succeed or fail together. Think of a bank transfer: debit one account and credit another - never only one of them.

Statement	Effect
`COMMIT`	Make all changes since the last commit permanent and visible to other sessions
`ROLLBACK`	Undo all changes since the last commit
`SAVEPOINT name`	Mark a point inside the transaction
`ROLLBACK TO name`	Undo only the changes made after the savepoint

```
BEGIN
    UPDATE accounts SET balance = balance - 500 WHERE id = 1;
    SAVEPOINT after_debit;
    UPDATE accounts SET balance = balance + 500 WHERE id = 2;
    COMMIT;                        -- both changes become permanent together
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;                  -- undo EVERYTHING since the last commit
        RAISE;                     -- re-raise so the caller knows it failed
END;
/
```

TIP `SELECT ... FOR UPDATE` (Chapter 23) and autonomous transactions (Chapter 25) build directly on these ideas. Remember: DDL statements such as `CREATE TABLE` commit implicitly.

CHAPTER 21

Dynamic SQL: Queries Built at Runtime

DEFINITION Sometimes you do not know the exact table or column names until the program is running (e.g. the user picks a table from a dropdown).

VISUALIZATION

```
Normal SQL (fixed at write-time):
   SELECT COUNT(*) FROM employees;     <- hard-coded

Dynamic SQL (decided at run-time):
   v_table := 'EMPLOYEES';             (could change based on input!)
   'SELECT COUNT(*) FROM ' || v_table  --> built as a STRING, then executed
```

```
DECLARE
   v_table_name VARCHAR2(30) := 'EMPLOYEES';
   v_count      NUMBER;
   v_sql_string VARCHAR2(200);
BEGIN
   v_sql_string := 'SELECT COUNT(*) FROM ' || v_table_name;
   EXECUTE IMMEDIATE v_sql_string INTO v_count;
   dbms_output.put_line('Row count: ' || v_count);
END;
/
```

Tool	When to use
EXECUTE IMMEDIATE	Simple dynamic queries - most common
DBMS_SQL	Complex cases where even the number of columns is unknown

21.1 Bind variables with USING

```
DECLARE
    v_count NUMBER;
BEGIN
    EXECUTE IMMEDIATE
        'SELECT COUNT(*) FROM employees WHERE department_id = :dept'
        INTO v_count
        USING 10;                    -- value is passed safely, not concatenated
    DBMS_OUTPUT.PUT_LINE(v_count);
END;
/
```

COMMON ERROR Never build dynamic SQL by concatenating **user-supplied values** into the string - that opens the door to **SQL injection** (Chapter 26). Use bind variables (`:name` + `USING`) for values, and `DBMS_ASSERT` for object names.

PART IV

Professional

Make PL/SQL fast, flexible and safe: bulk processing, advanced cursors, tuning, autonomous transactions, Oracle utilities and professional practices.

CHAPTER 22

Bulk Binding: Minimizing Context Switches

PL/SQL and SQL are executed by two separate engines. Every time your PL/SQL code runs a SQL statement, control switches from one engine to the other - a **context switch**.

THE PROBLEM

```
Without Bulk Binding (1000 rows, row-by-row):
PL/SQL engine <=> SQL engine  (switch #1)
PL/SQL engine <=> SQL engine  (switch #2)
...
PL/SQL engine <=> SQL engine  (switch #1000)   <- SLOW! 1000 context switches

With Bulk Binding:
PL/SQL engine <=> SQL engine  (ONE switch for all 1000 rows)  <- FAST!
```

Tool	Purpose
BULK COLLECT	Loads an ENTIRE result set into a collection in ONE operation
FORALL	Sends an ENTIRE array to the SQL engine for INSERT / UPDATE / DELETE in ONE operation

```
DECLARE
   TYPE t_emp_ids IS TABLE OF NUMBER;
   v_ids t_emp_ids;
BEGIN
   -- Read all IDs at once (no loop needed!)
   SELECT employee_id BULK COLLECT INTO v_ids FROM employees WHERE department_id = 10;

   -- Update all fetched IDs at once (loop-like syntax, but only ONE SQL engine call)
   FORALL i IN 1..v_ids.COUNT
      UPDATE employees SET salary = salary * 1.05 WHERE employee_id = v_ids(i);
END;
/
```

TIP For very large result sets, fetch in batches to protect memory: `FETCH c BULK COLLECT INTO v_ids LIMIT 1000;`

WARNING FORALL is **not** a loop - it is a single statement. It may contain only one DML statement.

CHAPTER 23

Advanced Cursors: Dynamic Results and Locking

23.1 REF CURSOR - a "pointer" to any result set

```
DECLARE
   TYPE ref_cursor_type IS REF CURSOR;
   v_cursor ref_cursor_type;
   v_name   VARCHAR2(50);
BEGIN
   OPEN v_cursor FOR SELECT first_name FROM employees WHERE department_id = 10;
   LOOP
      FETCH v_cursor INTO v_name;
      EXIT WHEN v_cursor%NOTFOUND;
      dbms_output.put_line(v_name);
   END LOOP;
   CLOSE v_cursor;
END;
/
```

Useful for passing results between PL/SQL and external applications (Java, Python). A ready-made weak type is also available: SYS_REFCURSOR.

23.2 SELECT ... FOR UPDATE - row locking

```
DECLARE
   CURSOR c_emp IS SELECT salary FROM employees WHERE employee_id = 100 FOR UPDATE;
   v_sal NUMBER;
BEGIN
   OPEN c_emp;
   FETCH c_emp INTO v_sal;
   -- row is LOCKED here; other users cannot modify it until commit/rollback
   UPDATE employees SET salary = v_sal + 100 WHERE CURRENT OF c_emp;
   CLOSE c_emp;
   COMMIT;
END;
/
```

> **TIP** Add NOWAIT (fail immediately if locked) or SKIP LOCKED (ignore locked rows) after FOR UPDATE to avoid waiting.

CHAPTER 24

Performance Tuning

Technique	What it does
NOCOPY hint	Passes `OUT` / `IN OUT` parameters by **reference** instead of copying - saves memory for large data
RESULT_CACHE	Oracle remembers a function's output for given inputs; skips re-execution next time
DBMS_PROFILER	Built-in "stopwatch" that shows which lines of code are slow
Bulk binding	Fewer context switches (Chapter 22)

RESULT_CACHE EXAMPLE

```
CREATE OR REPLACE FUNCTION get_dept_name(p_id NUMBER) RETURN VARCHAR2
RESULT_CACHE IS
   v_name VARCHAR2(50);
BEGIN
   SELECT department_name INTO v_name FROM departments WHERE department_id = p_id;
   RETURN v_name;
END;
/
```

NOCOPY EXAMPLE

```
CREATE OR REPLACE PROCEDURE process_big(p_data IN OUT NOCOPY t_big_collection) IS
BEGIN
    NULL;   -- p_data is passed by reference: no copy of a huge collection is made
END;
/
```

NOTE Measure before tuning. A profiler tells you where time is really spent; guessing usually optimizes the wrong thing.

CHAPTER 25

Advanced Capabilities and Oracle Utilities

Feature	Purpose
PRAGMA AUTONOMOUS_TRANSACTION	Creates an independent "side" transaction (e.g. log an error even if the main transaction rolls back)
Pipelined functions (PIPE ROW)	Stream rows back one at a time instead of returning a huge collection at once - saves memory
UTL_FILE	Read / write OS text files
DBMS_SCHEDULER	Automate / schedule background jobs
DBMS_CRYPTO	Encrypt sensitive data

25.1 Autonomous transactions

```
CREATE OR REPLACE PROCEDURE log_error(p_msg VARCHAR2) IS
   PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
   INSERT INTO error_log(message, log_date) VALUES (p_msg, SYSDATE);
   COMMIT;  -- commits ONLY this log entry, independent of the main transaction
END;
/
```

25.2 Pipelined functions

```
CREATE OR REPLACE TYPE t_num_tab AS TABLE OF NUMBER;
/

CREATE OR REPLACE FUNCTION gen_numbers(p_n NUMBER) RETURN t_num_tab PIPELINED IS
BEGIN
    FOR i IN 1..p_n LOOP
        PIPE ROW (i);        -- send one row to the caller immediately
    END LOOP;
    RETURN;
END;
/

SELECT * FROM TABLE(gen_numbers(5));
```

25.3 UTL_FILE (reading and writing OS files)

```
-- A DBA first creates a directory object:  CREATE DIRECTORY my_dir AS '/tmp/plsql';
DECLARE
    v_file UTL_FILE.FILE_TYPE;
BEGIN
    v_file := UTL_FILE.FOPEN('MY_DIR', 'hello.txt', 'W');
    UTL_FILE.PUT_LINE(v_file, 'Hello from PL/SQL');
    UTL_FILE.FCLOSE(v_file);
END;
/
```

25.4 DBMS_SCHEDULER (scheduling jobs)

```
BEGIN
    DBMS_SCHEDULER.CREATE_JOB(
        job_name        => 'NIGHTLY_CLEANUP',
        job_type        => 'PLSQL_BLOCK',
        job_action      => 'BEGIN DELETE FROM error_log WHERE log_date < SYSDATE - 30; COMMIT; END;',
        start_date      => SYSTIMESTAMP,
        repeat_interval => 'FREQ=DAILY; BYHOUR=2',
        enabled         => TRUE);
END;
/
```

25.5 DBMS_CRYPTO (encryption)

```
DECLARE
    v_key  RAW(32) := DBMS_CRYPTO.RANDOMBYTES(32);
    v_data RAW(2000) := UTL_I18N.STRING_TO_RAW('secret text', 'AL32UTF8');
    v_enc  RAW(2000);
BEGIN
    v_enc := DBMS_CRYPTO.ENCRYPT(
                src => v_data,
                typ => DBMS_CRYPTO.ENCRYPT_AES256 + DBMS_CRYPTO.CHAIN_CBC + DBMS_CRYPTO.PAD_PKCS5,
                key => v_key);
    DBMS_OUTPUT.PUT_LINE(RAWTOHEX(v_enc));
END;
/
```

WARNING These packages usually require explicit EXECUTE (or directory) privileges granted by a DBA. Never hard-code encryption keys in source code.

CHAPTER 26

Security and Professional Coding Practices

26.1 SQL injection

```
-- DANGEROUS: user input pasted into the statement
EXECUTE IMMEDIATE 'DELETE FROM employees WHERE name = ''' || p_name || '''';

-- SAFE: user input passed as a bind variable
EXECUTE IMMEDIATE 'DELETE FROM employees WHERE first_name = :n' USING p_name;
```

If p_name contains ' OR '1'='1, the dangerous version deletes every row. Bind variables treat input strictly as data, never as code.

26.2 Professional checklist

Practice	Why
Use %TYPE / %ROWTYPE	Declarations follow the table when columns change
Prefix names: v_ variables, c_ constants, p_ parameters, e_ exceptions	Avoid clashes with column names; code reads clearly
Prefer cursor FOR loops and bulk operations	Fewer bugs, fewer context switches
Handle expected exceptions specifically; WHEN OTHERS last	Do not hide real errors
Never swallow errors silently	At least log them (autonomous transaction) or re-raise
Use bind variables in dynamic SQL	Prevents SQL injection and speeds up parsing
Use explicit date literals / TO_DATE with a format	Independent of session settings
Use distinct names in nested blocks	Avoid shadowing confusion
Keep triggers small	Easier debugging
Commit deliberately, at transaction boundaries	Avoid half-finished data

CHAPTER A

Final Cheat Sheet

Concept	Keyword / syntax	Chapter
Block	DECLARE / BEGIN / EXCEPTION / END	2
Placeholder	name + datatype (variable, constant, record)	3
Variable	name datatype := value;	4
Anchored type	%TYPE	5
Constant	name CONSTANT type := value;	6
Literals	numeric, character, string, Boolean, date/time	7
Scope	nested blocks; inner sees outer, not the reverse	8
Operators	+ - * / **, = <> < > <= >=, AND OR NOT	9
DML in PL/SQL	INSERT / UPDATE / DELETE, SELECT INTO	10
Conditions	IF / ELSIF / ELSE, CASE	11
Loops	LOOP, WHILE, FOR, EXIT, CONTINUE	12
Row structure	%ROWTYPE / RECORD	13
Multi-row data	CURSOR, FOR loop	14
Error handling	EXCEPTION, RAISE_APPLICATION_ERROR	15
In-memory arrays	TABLE OF ... INDEX BY, nested table, VARRAY	16
Reusable code	PROCEDURE, FUNCTION	17
Grouped code	PACKAGE / PACKAGE BODY	18
Auto-fire on events	TRIGGER	19
Transactions	COMMIT, ROLLBACK, SAVEPOINT	20
Runtime SQL	EXECUTE IMMEDIATE	21
Mass operations	BULK COLLECT, FORALL	22
Flexible cursors	REF CURSOR, FOR UPDATE	23
Speed boosts	NOCOPY, RESULT_CACHE	24
Special tools	AUTONOMOUS_TRANSACTION, PIPE ROW, UTL_FILE, DBMS_SCHEDULER, DBMS_CRYPTO	25
Security	bind variables, least privilege	26

Complete path: Placeholders -> Variables -> Constants -> Scope -> Literals -> SQL in PL/SQL -> Control Flow -> Records -> Cursors -> Exceptions -> Collections -> Procedures / Functions -> Packages -> Triggers -> Dynamic SQL -> Bulk Binding -> Advanced Cursors -> Performance Tuning -> Advanced Oracle features.

Where to go next

Turn what you learned into practice: build a small **Employee Management System** that uses a package (`hr_pkg`) with procedures for hiring and raises, a cursor report, exception handling with custom errors, a salary-audit trigger and an error-logging autonomous transaction.